

Paterni Ponašanja

1 Strategy

Strategy je obrazac dizajna koji izdvaja algoritam iz matične klase i uključuje ga u posebne klase, čineći algoritme međusobno zamjenjivim neovisno od klijenta koji ih koristi.

U našem slučaju, možemo koristiti Strategy za generisanje kviza po oblastima, budući da sistem treba podržavati različite načine odabira pitanja ovisno o modu rada: priprema za kviz ili simulacija kviza.

- Definiranje interfejsa `IGenerisanjeKviza` s metodom `generisiPitanja(oblastID: Int, broj: Int): Pitanje[]`.
- Implementacija klase `NasumicniOdabirStrategy` koja nasumično bira pitanja uz ravnomjernu distribuciju prema težini za simulaciju kviza.
- Implementacija klase `SekvencijalniOdabirStrategy` koja prikazuje sva pitanja redom za mod pripreme za kviz.

Korištenje: `KvizController` prima odgovarajuću strategiju i delegira generisanje pitanja konkretnoj implementaciji bez poznavanja detalja algoritma. Zamjena strategije ne zahtijeva izmjenu kontrolera.

2 State

State je obrazac dizajna koji mijenja način ponašanja objekta na osnovu trenutnog stanja, tako da objekat izgleda kao da mijenja svoju klasu ovisno o stanju u kojemu se nalazi.

U našem slučaju, možemo koristiti State za upravljanje tokom kviz sesije koja prolazi kroz više stanja: `AKTIVAN`, `U_TOKU` i `ZAVRSEN`, pri čemu svako stanje dozvoljava drugačiji skup operacija.

- Definiranje interfejsa `IStatusSesije` s metodama `pokreni(): void`, `odgovori(): void` i `zavrsi(): void`.
- Implementacija konkretnih stanja `AktivnoStanje`, `UTokuStanje` i `ZavrsonoStanje` koji definišu dozvoljene operacije za svako stanje sesije.
- Klasa `KvizSesija` drži referencu na trenutno stanje i delegira pozive odgovarajućem stanju.

Korištenje: Kada student pokrene kviz, sesija prelazi iz `AKTIVAN` u `U_TOKU` stanje. Po isteku vremena ili predaji odgovora, sesija automatski prelazi u `ZAVRSEN` stanje i onemogućava dalje odgovaranje.

3 Template Method

Template Method je obrazac dizajna koji omogućava algoritmima da izdvoje pojedine korake u podklase, definirajući kostur algoritma u baznoj klasi dok konkretne podklase implementiraju varijabilne korake.

U našem slučaju, možemo koristiti Template Method za procesiranje feedbacka, budući da i `PrijedlogPitanja` i `PrijavaGreske` dijele isti opći tok obrade ali se razlikuju u koracima validacije.

- Apstraktna klasa `Feedback` definira metodu `procesiraj()` koja poziva korake: `validiraj()`, `pohrani()` i `obavijesti()`.
- `PrijedlogPitanja` override-uje `validiraj()` da provjeri format prijedloga pitanja i ponuđenih odgovora.
- `PrijavaGreske` override-uje `validiraj()` da provjeri da li je naveden opis greške i referentno pitanje.

Korištenje: `FeedbackController` poziva metodu `procesiraj()` na primljenom `feedback` objektu bez poznavanja konkretnog tipa, dok svaka podklasa primjenjuje svoju validaciju, a pohrana i slanje obavijesti ostaju zajednički.

4 Chain of Responsibility

Chain of Responsibility je obrazac dizajna koji predstavlja listu objekata za obradu zahtjeva — ukoliko objekat ne može da odgovori na zahtjev, prosljeđuje ga narednom objektu u nizu.

U našem slučaju, možemo koristiti Chain of Responsibility za obradu feedbacka koji pristiže od studenata. Prijedlozi pitanja i prijave grešaka mogu proći kroz lanac provjera i odobravanja od strane Moderadora i Administratora.

- Definiranje apstraktnog handlera `FeedbackHandler` s metodom `obradi(feedback: Feedback): void` i referencom na sljedećeg handlera u nizu.
- Implementacija `ModeratorHandler` koji obrađuje feedbacke iz svoje oblasti, a ostale prosljeđuje dalje u lancu.
- Implementacija `AdministratorHandler` koji prima neobrađene feedbacke od Moderadora i donosi konačnu odluku.

Korištenje: Kada student pošalje feedback, zahtjev ulazi u lanac. Moderator obrađuje prijedloge unutar svoje oblasti, dok se zahtjevi koji zahtijevaju više ovlaštenje automatski prosljeđuju Administratoru bez potrebe za ručnim preusmjerenjem.

5 Command

Command je obrazac dizajna koji pretvara zahtjev u samostalni objekat, omogućavajući parametrizaciju klijenata različitim zahtjevima, njihovo stavljanje u red čekanja, logovanje kao i podršku za operacije poništavanja (`undo/redo`).

U našem slučaju, možemo koristiti Command za akcije upravljanja sadržajem kao što su dodavanje, izmjena i brisanje pitanja i predmeta od strane Moderadora ili Administratora.

- Definiranje interfejsa `IKomanda` s metodama `izvrsi(): void` i `ponisti(): void`.
- Implementacija konkretnih komandi `DodajPitanjeKomanda`, `IzmijeniPitanjeKomanda` i `ObrisiPitanjeKomanda`.
- Klasa `PitanjeController` igra ulogu invoker-a koji poziva komande i čuva historiju izvršenih akcija u listi.

Korištenje: Svaka akcija nad sadržajem kreira odgovarajući komandni objekat koji se izvršava i čuva u historiji. Na taj način sve izmjene su evidentirane, a sistem može podržati poništavanje posljednje akcije bez promjene logike u kontrolerima.

6 Iterator

Iterator je obrazac dizajna koji omogućava pristup elementima kolekcije sekvencijalno bez poznavanja interne strukture kolekcije.

U našem slučaju, možemo koristiti Iterator za prolazak kroz pitanja unutar kviz sesije tokom simulacije ili pripreme, čime se apstrahuje način na koji su pitanja pohranjena i organizirana unutar `KvizSesija`.

- Definiranje interfejsa `IPitanjeIterator` s metodama `hasNext(): Boolean` i `next(): Pitanje`.
- Implementacija `KvizIterator` koji prolazi kroz listu pitanja unutar `KvizSesija` uzimajući u obzir redni broj iz `KvizSesijaPitanje`.
- Klasa `KvizSesija` implementira metodu `getIterator()` koja vraća odgovarajući iterator za tekuću sesiju.

Korištenje: Prikaz pitanja tokom kviza koristi iterator koji sekvencijalno isporučuje pitanja jedno po jedno, neovisno o tome da li su pohranjena kao lista, niz ili direktno iz baze podataka.

7 Mediator

Mediator je obrazac dizajna koji enkapsulira protokol za komunikaciju među objektima dozvoljavajući da objekti komuniciraju bez međusobnog poznavanja interne strukture, čime se smanjuje sprežanost između komponenti sistema.

U našem slučaju, možemo koristiti Mediator za koordinaciju između kontrolera sistema koji moraju međusobno razmjenjivati informacije, poput koordinacije između `KvizController`-a, `LeaderboardController`-a i `FeedbackController`-a nakon završetka kviz sesije.

- Definiranje interfejsa `IKvizMediator` s metodom `notify(posiljalac: Object, dogadaj: String): void`.
- Implementacija klase `KvizMediator` koja prima obavijesti od kontrolera i koordinira odgovarajuće akcije između komponenti.
- Kontroleri `KvizController` i `LeaderboardController` čuvaju referencu na mediator i komuniciraju isključivo kroz njega.

Korištenje: Nakon što `KvizController` završi sesiju, obavještava `KvizMediator` koji zatim koordinira ažuriranje leaderboarda i slanje obavijesti studentu, bez direktne veze između kontrolera.

8 Observer

Observer je obrazac dizajna koji uspostavlja relaciju između objekata takvu da kada se stanje jednog objekta promijeni, svi vezani objekti dobiju informaciju i budu automatski ažurirani.

U našem slučaju, možemo koristiti Observer za automatsko obavještavanje korisnika o relevantnim događajima kao što su promjena statusa feedbacka, odobravanje prijedloga pitanja ili dostupnost novih sadržaja na platformi.

- Definiranje interfejsa `IObserver` s metodom `update(događaj: String): void` koji implementiraju sve klase koje žele primati obavijesti.
- Implementacija klase `ObavijestSubject` koja čuva listu pretplatnika i poziva metodu `notifyObservers()` kada nastane relevantni događaj.
- Klase `Student`, `Moderator` i `Administrator` implementiraju `IObserver` i registruju se za događaje relevantne za svoju ulogu.

Korištenje: Kada `Administrator` odobri prijedlog pitanja ili promijeni status feedbacka, `ObavijestSubject` automatski obavještava sve registrovane posmatrače i šalje obavijesti putem klase `Obavijest` i eksternog MAIL servisa.

9 Visitor

Visitor je obrazac dizajna koji definiše i izvršava nove operacije nad elementima postojeće strukture ne mijenjajući samu strukturu, enkapsulacijom operacija u posebnu klasu posjetitelja.

U našem slučaju, možemo koristiti Visitor za izvođenje različitih operacija nad pitanjima i odgovorima bez izmjene tih klasa, poput izvoza pitanja u različite formate ili generisanja statistike po težini i oblasti.

- Definiranje interfejsa `IPitanjeVisitor` s metodama `visitPitanje(p: Pitanje): void` i `visitOdgovor(o: Odgovor): void`.
- Implementacija `StatistikaVisitor` koji prikuplja podatke o distribuciji pitanja prema težini i oblasti bez izmjene klasa modela.
- Implementacija `EksportVisitor` koji formatira pitanja i odgovore za izvoz u eksterni format.

Korištenje: `PitanjeController` može primijeniti bilo koji Visitor na kolekciju pitanja bez izmjene klase `Pitanje`. Dodavanje novih operacija zahtijeva samo kreiranje novog Visitor-a, a ne modifikaciju postojećih klasa.

10 Interpreter

Interpreter je obrazac dizajna koji podržava interpretaciju instrukcija napisanih za određenu upotrebu, definirajući gramatiku jednostavnog jezika i interpreter koji obrađuje izraze tog jezika.

U našem slučaju, možemo koristiti Interpreter za parsiranje i evaluaciju izraza filtriranja ili pretraživanja pitanja po kriterijima poput oblasti, težine i predmeta, omogućavajući fleksibilno pretraživanje baze pitanja.

- Definiranje interfejsa `IIzraz` s metodom `interpretiraj(kontekst: KontekstPretrage): Boolean`.
- Implementacija terminalnih izraza `OblastIzraz` i `TezineIzraz` koji evaluiraju jednostavne uvjete pretrage.
- Implementacija kompozitnih izraza `IIzraz` koji kombinuju više uvjeta logičkim operatorima `AND` i `OR` za složene pretrage.

Korištenje: Korisnik definira kriterije pretrage pitanja, sistem ih parsira u stablo izraza i evaluira nad bazom pitanja. Na taj način logika pretrage je fleksibilna i proširiva bez izmjene klasa `Pitanje` ili `PitanjeController-a`.

11 Memento

Memento je obrazac dizajna koji omogućava spašavanje internog stanja nekog objekta van sistema i njegovo ponovno vraćanje u prethodno stanje, bez narušavanja enkapsulacije objekta.

U našem slučaju, možemo koristiti Memento za čuvanje stanja kviz sesije, omogućavajući vraćanje na prethodno pitanje ili oporavak sesije u slučaju tehničke greške ili prekida veze tokom simulacije kviza.

- Kreiranje klase `KvizSesijaMemento` koja čuva snapshot stanja `KvizSesija`: trenutno pitanje, dosad date odgovore i preostalo vrijeme.
- Klasa `KvizSesija` implementira metode `sacuvajStanje(): KvizSesijaMemento` i `vratiStanje(m: KvizSesijaMemento): void`.
- Klasa `KvizCaretaker` čuva historiju memento objekata i upravlja vraćanjem na prethodno stanje po potrebi.

Korištenje: Sistem periodično ili na zahtjev čuva stanje aktivne kviz sesije putem `KvizCaretaker-a`. U slučaju prekida veze ili greške, student može nastaviti sesiju od mjesta gdje je stao, bez gubitka dosad datih odgovora.